

实验报告

课程：数字逻辑与计算机组成实验

实验十：RV32I 流水线 CPU

姓名： 郑袭明

学号： 251502027

日期：2026 年 6 月 4 日

目录

1	设计思路与架构差异	2
1.1	五级流水线架构与时钟调整	2
1.2	流水线冲突解决机制	2
1.3	斐波那契程序机器码调整与前瞻取指	2
1.4	时序收敛与综合结果对比	3
2	实验总结与反思	3

1 设计思路与架构差异

1.1 五级流水线架构与时钟调整

本实验将 Lab 9 的单周期 CPU 升级为标准的五级流水线 CPU，划分取指、译码、执行、访存和写回五个阶段。在各阶段之间引入了流水线寄存器以缓存数据和控制信号（如 PC_ID、instr、PC_EX、val1_EX、alurestult_MEM 等）。

同时，时钟与读写逻辑作出了如下调整：

1. **寄存器堆**：由 Lab 9 的下降沿写回改为上升沿写回，并实现了前推逻辑，在读写同一寄存器时直接输出写入值，以解决写回阶段到译码阶段的数据冲突。
2. **存储器时钟**：指令存储器时钟 imemclk 改为 clock（同相），以配合流水线节拍。

1.2 流水线冲突解决机制

流水线引入了以下两种冲突解决机制：

- **数据冲突**：
 - **前推机制**：在执行阶段引入前推单元。若检测到当前源寄存器与访存或写回阶段的目标寄存器相同，则直接将访存阶段计算结果或写回数据前推给 ALU 输入端，避免了绝大多数 RAW 冲突引起的暂停。
 - **暂停机制**：对于 Load-Use 冲突（即上一条是 Load 指令，且当前指令需要读取其加载的寄存器），由于数据在访存阶段结束后才能获得，无法通过前推完全解决。此时通过暂停单元拉高 stall 信号，保持 PC_IF 和 PC_ID 不变，并在译码/执行流水线寄存器中清零控制信号以插入一个 NOP。
- **控制冲突**：分支与跳转决策在执行阶段完成。当检测到跳转发生时，需清空已被读入但失效的指令，即向译码阶段与执行阶段的流水线寄存器中强制写入 NOP，实现 2 个周期的流水线冲刷。

1.3 斐波那契程序机器码调整与前瞻取指

由于分支跳转延迟的存在，CPU 会在执行阶段跳转结果确定前，前瞻性地预取后续的 2 条指令。

顶层模块 top 的停机逻辑通过监测指令总线上的自定义停机码 32'hdead10cc 来触发挂起状态。若将停机码紧跟在 jal 之后放置（如 Lab 9 中的设计，位于 0x08），当 jal 指令尚在译码与执行阶段时，停机码已被预取并呈现在指令存储器输出总线上。这将错误地触发顶层模块的停机状态机，导致程序未进入斐波那契循环就已停机。

因此，在 `fib.coe` 中进行了如下调整：

1. 在 `jal` 指令之后插入了 2 条 `nop` 指令（即在 `0x08` 和 `0x0c` 填入 `32'h00000013`），将停机码推迟到 `0x10` 处。
2. 将 `jal` 的跳转偏移量由 12 修正为 16（目标地址变为 `0x14`）。
3. 当子程序通过 `jalr` 返回时，跳转至保存的返回地址 `0x08`，顺序执行 2 个 `nop` 指令后，于 `0x10` 正常读取并执行停机码。

1.4 时序收敛与综合结果对比

由于流水线结构将单周期的长组合路径（寄存器堆 → ALU → 数据存储器 → 寄存器堆）拆分为五个独立阶段，关键路径被极大地缩短。

对比编译报告 `tight_setup_hold_pins.txt`：

- **Lab 9 单周期 CPU**：存在大量时序极度紧张的引脚（日志大小约 3.9 KB），时序裕量极低。
- **Lab 10 流水线 CPU**：仅剩 1 个时序紧张引脚（`cpu/PC_IF_reg[0]/D`，日志仅 815 字节），整体建立与保持时间裕量显著提升，大幅提高了主频上限。

2 实验总结与反思

流水线不再需要上升沿写，下降沿读，但我最开始写的时候忘记改这里了，就造成了时序问题；把时钟改一下就好了。

斐波那契程序部分，由于停机指令会被提前读取，我最开始实验的时候程序无法输出正确的结果（只能原样输出输入的数字）。我始终没发现这个问题，后来问了 AI 才意识到这个点，加了两个 `nop` 就可以了。查询有关资料，可以知道在跳转指令后加 `nop` 是很常见的，这里的 `nop` 被称为分支延迟槽，可以防止程序逻辑出错。

把单周期和流水线相对比，在没有跑过综合的时候我是没想到时序紧张程度会差这么多（我中间有一次综合的结果里，一个时序紧张的引脚都没有），这是我没想到的。

总的来说，很多问题是我在做实验的时候才能发现的，之前没有亲自上手做真的体会不到。也算是特别的收获吧。